

로봇 이동

△ 삼성 B형 시험 후 기억으로 복원한 연습판입니다. 알고리즘 구조(방향 상태 BFS, out_mask, 경유지 순열)는 코드로 검증했으나 좌표계·교차로 규칙의 세부는 원문과 다를 수 있습니다.

시간 제한 : 25개 테스트케이스를 합쳐서 Python의 경우 8초

메모리 제한 : 표준 | **제출** : Main 고정 + User Code (init / build / move) 구현

문제

크기 $N \times N$ 의 격자 도시가 있다. 처음에는 건물과 도로가 존재하지 않는다.

건물을 추가하면 건물의 바깥쪽을 한 칸 두께로 둘러싸는 도로가 함께 생성된다. 건물은 직사각형이며, 좌상단 좌표 (mX, mY) , 너비 mW , 높이 mH 를 가진다. 건물 내부 영역은 $x = mX \sim mX+mW-1$, $y = mY \sim mY+mH-1$ 이고, 그 바깥 한 칸이 도로가 된다. 출입구는 건물 내부 가장자리 칸 하나이며, 건물 내부 기준 상대 좌표 $(mDoorX, mDoorY)$ 로 주어진다.

건물끼리는 서로 겹치거나 붙을 수 없다. 그러나 건물 바깥쪽 도로는 서로 겹칠 수 있으며, 두 건물의 도로가 여러 칸을 공유할 수도 있다.

[Fig. 1] 건물 하나와 그 둘레 도로



파란 화살표 = 시계방향 도로, 주황 ● = 교차로, 초록 D = 출입구

도로 이동 규칙

로봇은 건물 주위의 도로를 그 건물 기준 시계방향으로만 이동한다. 윗변은 $\rightarrow$, 오른쪽 변은 $\downarrow$, 아랫변은 $\leftarrow$, 왼쪽 변은 $\uparrow$ 방향이다. 도로 한 칸을 이동하는 비용은 1이며, 일반 도로 중간에서는 방향을 바꿀 수 없다.

교차로

모든 건물 도로 사각형의 네 꼭짓점은 교차로이다. 한 교차로에 여러 건물의 도로가 함께 있을 수 있다. 교차로에서는 직진 / 좌회전 / 우회전 / U턴이 가능하지만, 선택한 방향으로 실제 도로가 존재하고 그 도로가 어떤 건물의 시계방향 도로여야 한다. 즉 교차로에 도착해야만 다른 건물의 도로로 갈아탈 수 있다.

출입구

출입구를 드나들 때는 항상 우회전만 가능하다. 즉 로봇의 진행 방향 기준 오른쪽 한 칸에 출입구가 있을 때만 그 출입구로 들어갈 수 있다. 도로 $\rightarrow$ Door, Door $\rightarrow$ 도로 비용은 각각 1이며, 건물 내부에서는 별도의 이동을 하지 않는다.

함수 명세

- `init(N)` : 크기 $N \times N$ 의 빈 도시를 생성한다.
- `build(mID, mX, mY, mW, mH, mDoorX, mDoorY)` : 건물을 추가한다. 건물 ID는 중복되지 않으며, 호출 횟수는 최대 약 1000이다.
- `move(mStart, mEnd, M, mID)` : `mStart` 출입구에서 출발하여, `mID[0:M]`의 모든 경유 건물 출입구를 방문한 뒤(순서 자유, $M \leq 5$) `mEnd` 출입구까지 가는 **최소 이동 거리**를 반환한다. 이동 불가능한 경우는 주어지지 않는다.

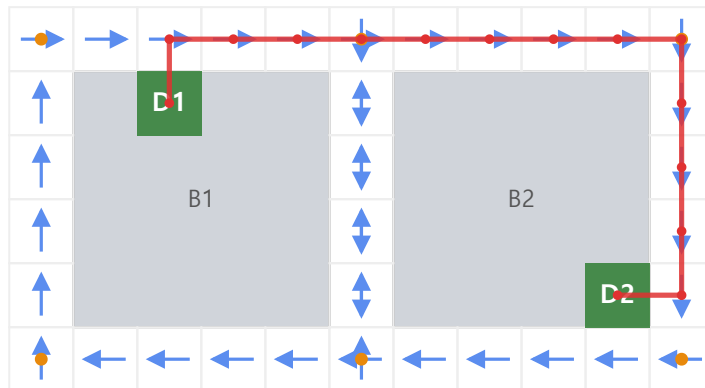
예시

아래 [Table 1]과 같이 함수가 호출되는 경우를 생각해 보자.

Order	Function	Return
1	<code>init(15)</code>	
2	<code>build(1, 3, 3, 4, 4, 1, 0)</code>	
3	<code>build(2, 8, 3, 4, 4, 3, 3)</code>	
4	<code>move(1, 2, 0, {})</code>	14
5	<code>move(2, 1, 0, {})</code>	20
6	<code>move(1, 1, 0, {})</code>	0

건물 1은 출입구가 윗변 가운데(D1), 건물 2는 아랫변(D2)에 있다. 두 건물의 도로는 $x = 7$ 열을 공유한다. 로봇이 D1에서 나와 시계방향으로 돌다가 공유 교차로에서 건물 2의 도로로 갈아타 D2로 진입하면 **14칸**이다([Fig. 2]). 반대로 D2→D1은 시계방향 제약 때문에 경로가 달라 **20칸**이 된다.

[Fig. 2] D1 출발 -> D2 도착 (빨간 경로 = 14칸)



빨간 선 = D1→D2 최단경로(14칸). 각 점은 지나는 도로 칸.

입력 형식 (Main)

첫 줄에 테스트케이스 수 T 와 배점 `MARK`. 각 테스트케이스는 명령 수 Q 와 Q 개의 명령으로 구성된다.

```
INIT   : 100 N
BUILD  : 200 mID mX mY mW mH mDoorX mDoorY
MOVE   : 300 mStart mEnd M (mID x M) expected
```

모든 `move`의 반환값이 `expected`와 일치하면 그 테스트케이스는 `MARK`점, 아니면 0점이다.

제약 조건

- 격자 크기 N , 건물 수($\leq$ 약 1000) — 건물끼리 겹치거나 붙지 않는다.
- 경유지 $M \leq 5$ (방문 순서 순열 $5! = 120$ 이내).

- 도로는 방향 그래프이므로 $\text{dist}(A,B) \neq \text{dist}(B,A)$ 가 될 수 있다.

풀이 핵심

- **상태에 방향을 넣는다** — `visited[y][x][dir]` . 같은 칸도 진행 방향이 다르면 다른 상태다.
- **도로를 `out_mask[y][x]` 비트로 저장한다.** 그 칸에서 시계방향으로 나갈 수 있는 방향 집합이다. 반대 방향 도로가 겹쳐도 두 비트가 함께 켜져 자연스럽게 처리된다. 일반 도로에서는 현재 방향만, 교차로에서는 켜진 모든 방향으로 분기한다.
- **출입구 진입** — 진행 방향의 오른쪽 $(\text{dir}+1)\%4$ 칸이 목표 Door면 +1로 진입한다.
- **경유지** — 관련 출입구들 사이의 Door→Door 최단거리를 BFS로 구한 뒤, 경유지 순열(≤ 120)을 돌려 최소 합을 찾는다.